Software Engineering

CSC648/848

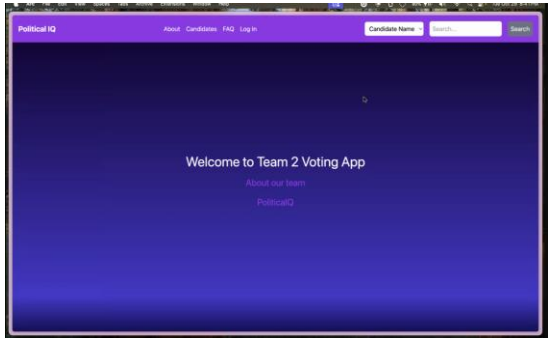
M3

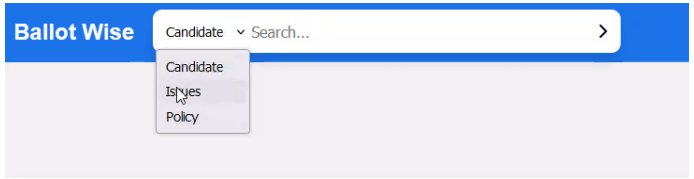
Functionality and pages/use cases as below:
- Home page
- Search (including search field validation)
- Search results
- Filtering
- Search Details and maps (if applicable)
- Messaging/contact seller/user (if applicable)
- Data Upload dashboards (user, admin)
- UI responsiveness (resize the browser)
- Performance (e.g. display of results list)

Brief review of **code, github, database** etc.

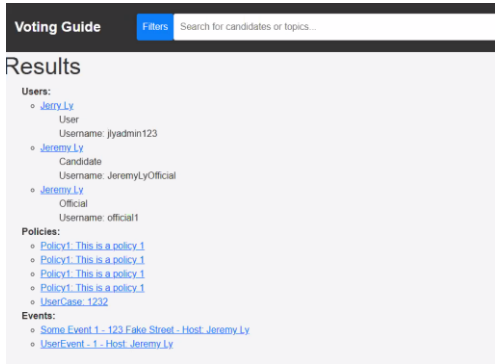
Project status – be ready to verbally explain status of the issues below (Teamwork, Risks, Coding practices, SE Code Management, Security coding best practices, digital content)

List of P1 features committed for delivery agreed at M3 review meeting

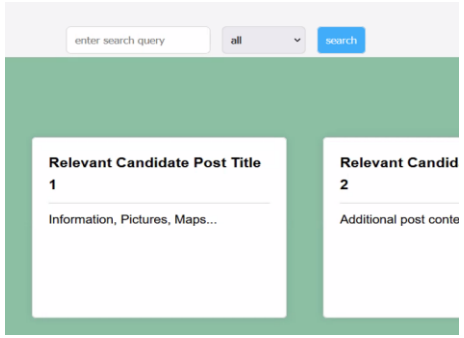

Political IQ
About Candidates FAQ Log In
Candidate Name
Search
Welcome to Team 2 Voting App
About our team
PoliticalIQ


Ballot Wise
Candidate
Search...
Candidate
Issues
Policy

Home
Candidate Dashboard
Search for a Candidate
Welcome, johnsmith
Logout
About Us
VOTE
InstaVoteTok
Welcome to InstaVoteTok.


Voting Guide
Filters
Search for candidates or topics...
Results
Users:
Jerry Ly
User
Username: jlyadmin123
Jeremy Ly
Candidate
Username: JeremyLyOfficial
Jeremy Ly
Official
Username: official1
Policies:
Policy1: This is a policy 1
Policy1: This is a policy 1
Policy1: This is a policy 1
Policy1: This is a policy 1
UserCase: 1232
Events:
Some Event 1 - 123 Fake Street - Host: Jeremy Ly
UserEvent - 1 - Host: Jeremy Ly

Voting Guide
Helping you find the Ideal Candidate
Enter a search term
Candidate
Event
Policies
Election Info
Political Party
Polling Station


enter search query
all
search
Relevant Candidate Post Title 1
Information, Pictures, Maps...
Relevant Candid 2
Additional post conte

M3 Feedback

Final Project Main Functions

- <u>Web App: Voting Guide with AI+</u>

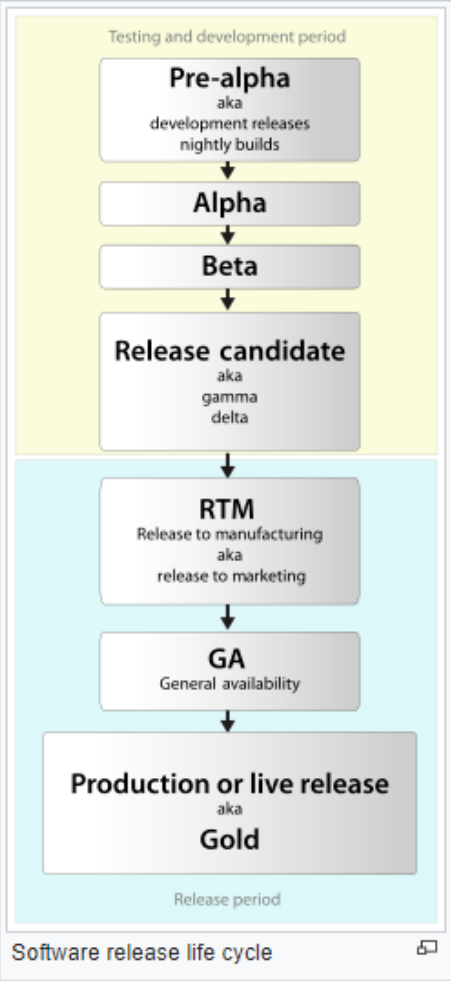
1. **Search & Filter** by state, political party and issues/policies.
2. **Enter** official information (political parties, voter registration, polling location, voting process, etc.)
3. Site **Registration** (College Students, General public, Officials, Admin, etc.).
4. **Rate** candidates.
5. Search results may be displayed **using maps**
6. **AI capabilities** is a plus.

Two entrepreneurs, SFSU alumni, decided to create a start-up comprising of a team of about 6+ students to develop a web-based service to help USA college students and general public to identify *county, state, and federal*/national **candidates** running for public offices and their respective *policies*, and *background*. The web application will allow officials from the different **political parties** to enter the corresponding official information, and the **Elections office** to enter information about voter registration, polling locations, and voting process. College students and general public should be able to search by state, political party and issues/policies so results can list the corresponding data, and at the same time; user (student and general public) can enter their issues of interest and the AI chatbot can assist them with mapping what candidates are aligned with the user interest. This application is targeting three primary audiences: (1) College Students, (2) General Public, and (3) Officials from the political parties and election office. Students and general public users should be able to rate candidates. Officials from political parties and election office may need to be registered to use all the functionality of the application. General users may access to limited search capabilities and application functionality. Search results may display maps.

In Perspective: Where we are?

In Perspective:
M1: Use Cases, High Level Requirements and Architecture (Personas and Use Cases)
M2: More Detailed Requirements, Specs, Architecture, UI mock-ups and Vertical SW prototype
M3: Review of functionality, UI, SW and planning for final product delivery
M4: Beta Launch, QA and Usability Testing and Final Commitment for Product Features (P1 list)
M5: Final Demo and Documents for Grading

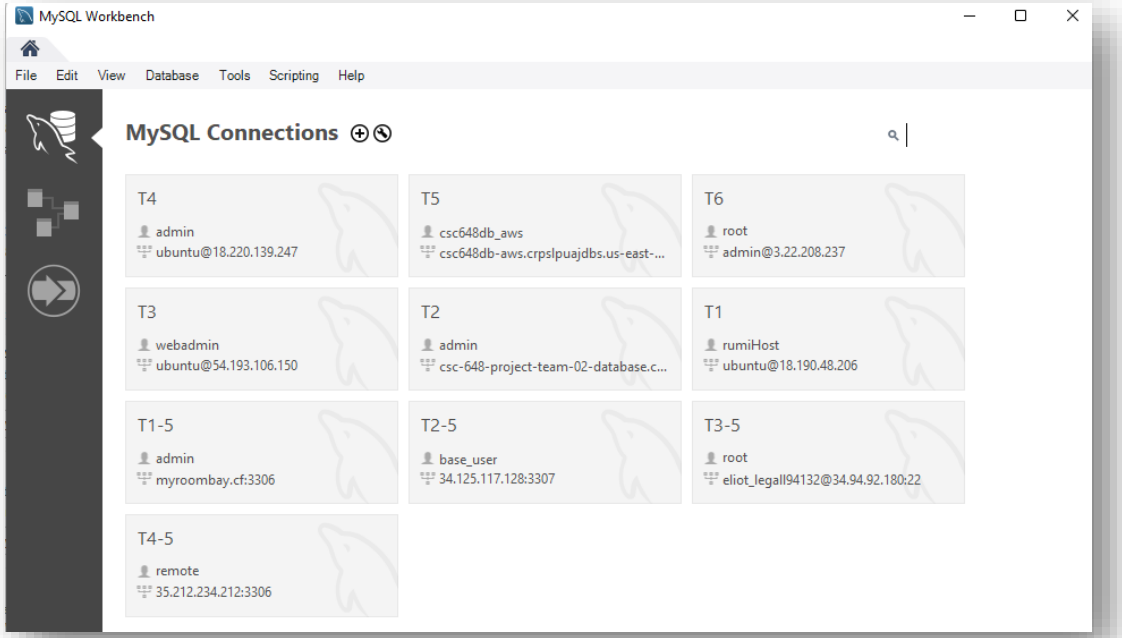
1. Make sure the code is working on the prod server
2. Full working functionalities (use cases) – front-end and back-end
3. Rating functionality is still pending
4. Finalize the filtering functionality
5. Finalize P1 List
6. Look and feel, need work

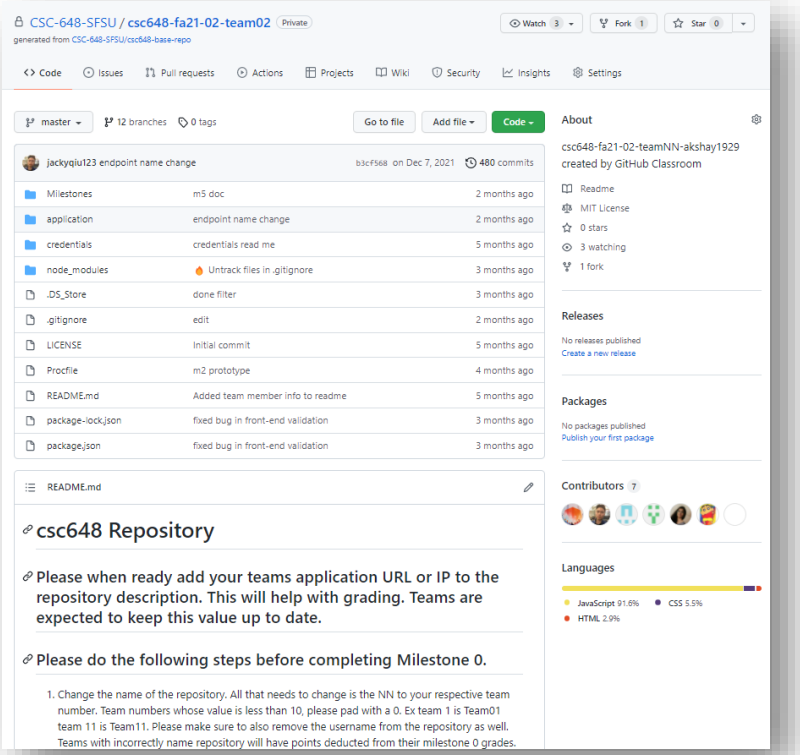


M0 Expectation

All projects will be inspected and graded on the due date. The key for grading will be your team's:

1. Correct installation and configuration of software stack,
2. Correct setup and usage of GitHub, and the
3. Deployment of your About Me Web Application





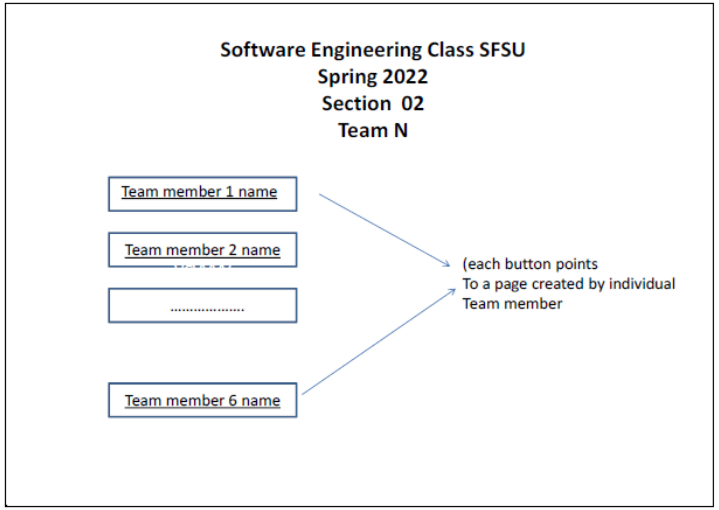
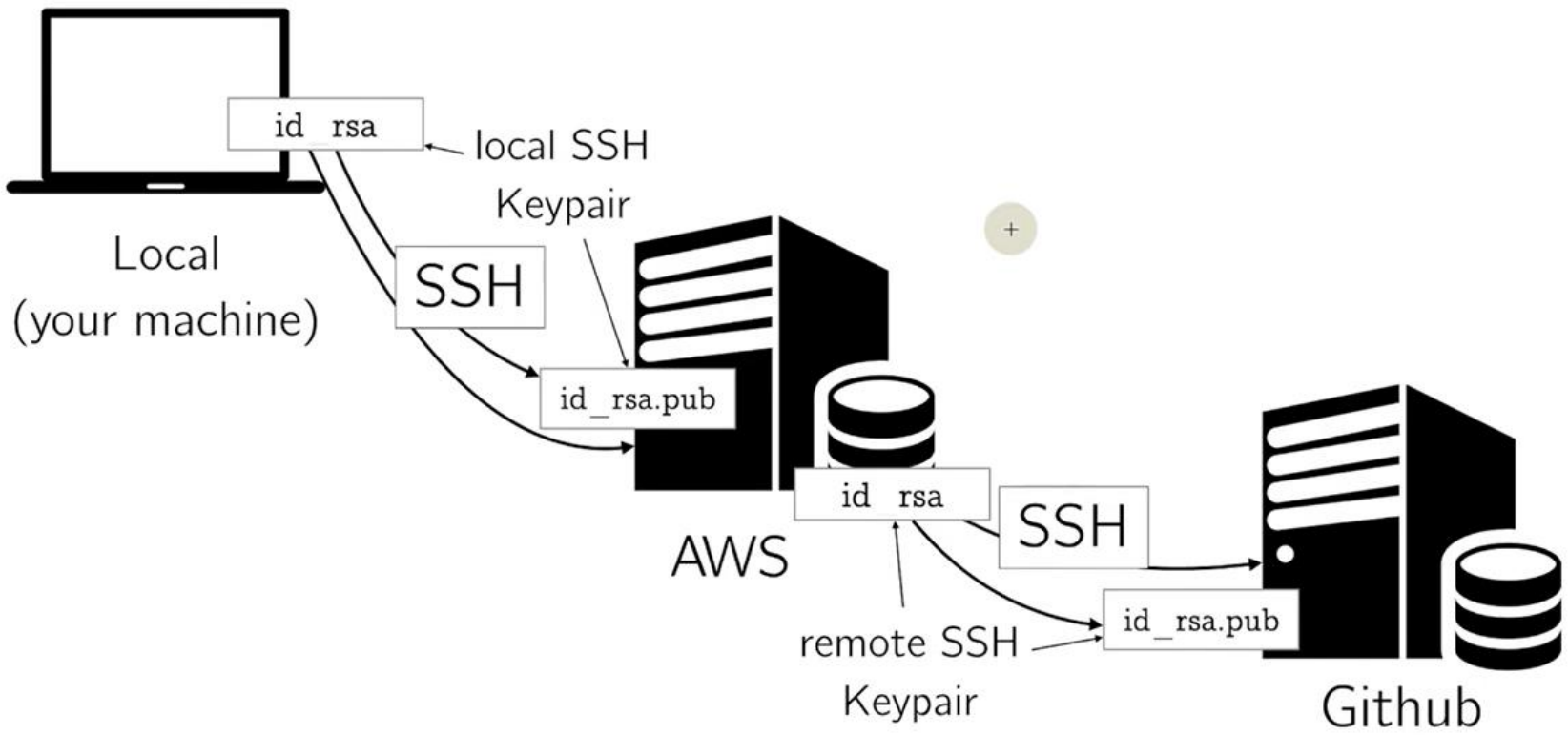


Figure 1. Example mockup of Milestone 0 team home page.

Every single student should have this conf…



Class Plan

- **LECTURE.** SE Methodology and Process Lifecycle
- **Team Collaboration.**
  - (Work on M4 documentation)

Henry
VILLAR

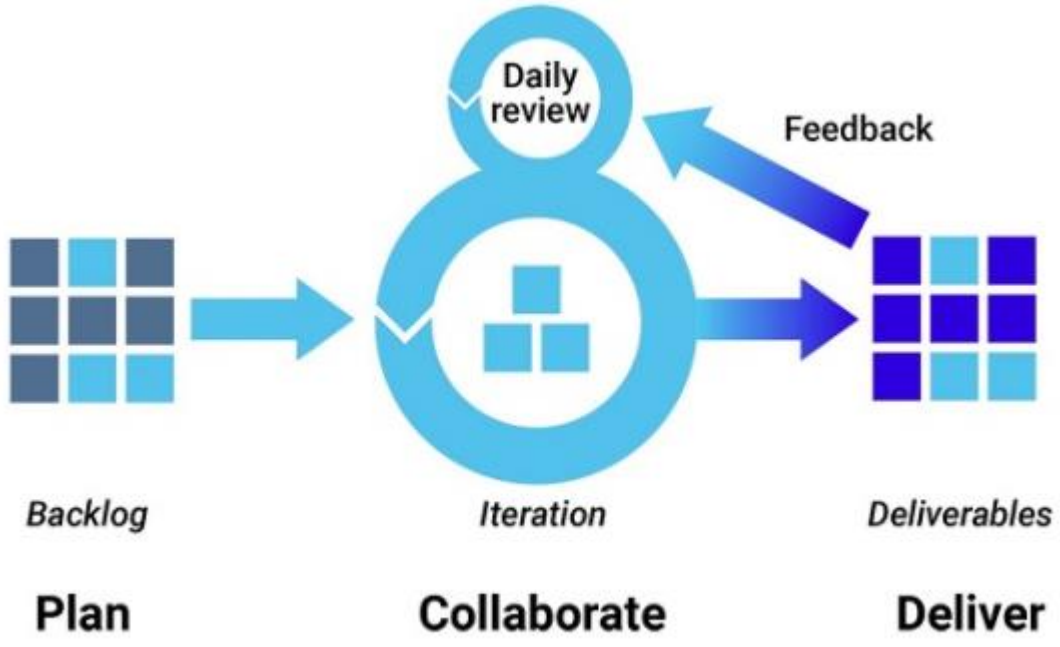
SE METHODOLOGY AND PROCESS

Lifecycle

SW Development Methodologies: Top 4

Agile development methodology

Teams use the agile development methodology to minimize risk (such as bugs, cost overruns, and changing requirements) when adding new functionality. In all agile methods, teams develop the software in iterations that contain mini-increments of the new functionality. There are many different forms of the agile development method, including scrum, crystal, extreme programming (XP), and feature-driven development (FDD).



Source: https://www.synopsys.com/blogs/software-security/top-4-software-development-methodologies/

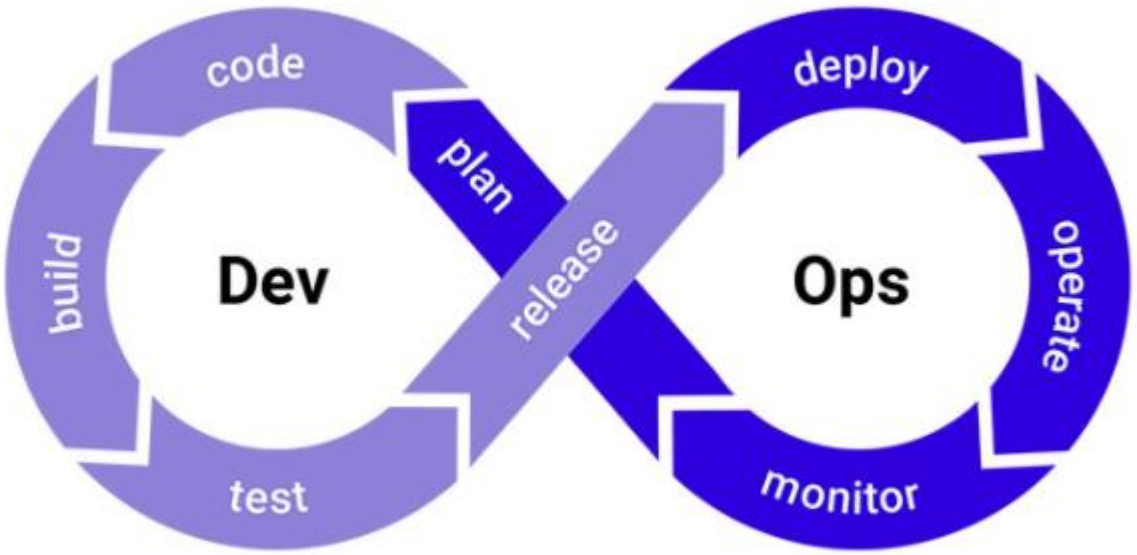
Agile Pros and Cons

- **Pros:** The primary benefit of agile software development is that it allows software to be released in iterations. Iterative releases improve efficiency by allowing teams to find and fix defects and align expectation early on. They also allow users to realize software benefits earlier, with frequent incremental improvements.
- **Cons:** Agile development methods rely on real-time communication, so new users often lack the documentation they need to get up to speed. They require a huge time commitment from users and are labor intensive because developers must fully complete each feature within each iteration for user approval.
- Agile development methods are similar to rapid application development (see below) and can be inefficient in large organizations. Programmers, managers, and organizations accustomed to the waterfall method (see below) may have difficulty adjusting to an agile SDLC. So a hybrid approach often works well for them.

SW Development Methodologies: Top 4

DevOps deployment methodology

DevOps is not just a development methodology but also a set of practices that supports an organizational culture. DevOps deployment centers on organizational change that enhances collaboration between the departments responsible for different segments of the development life cycle, such as development, quality assurance, and operations.



Source: https://www.synopsys.com/blogs/software-security/top-4-software-development-methodologies/

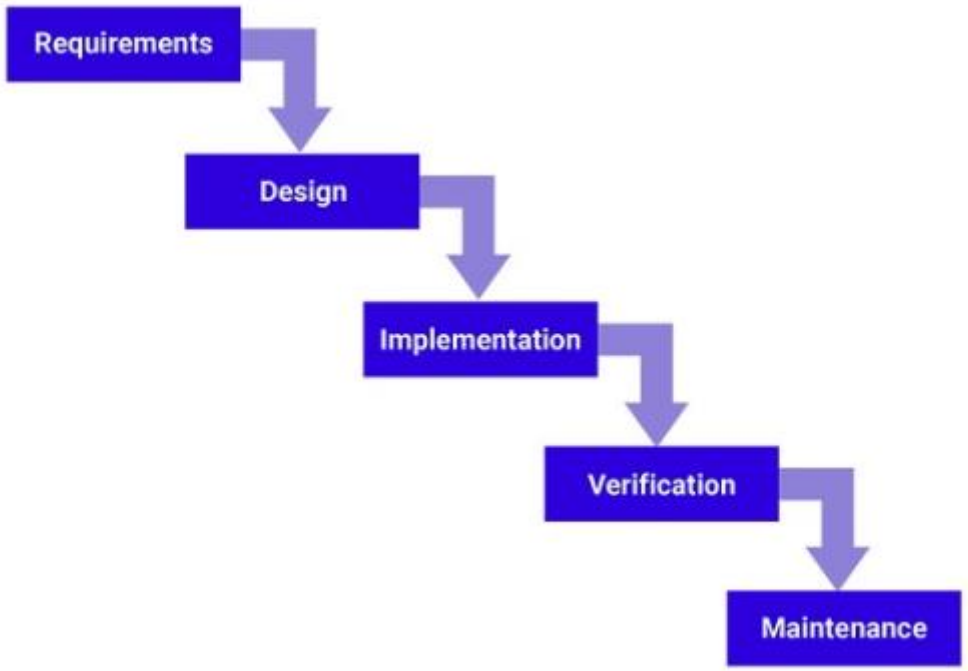
DevOps Pros and Cons

- **Pros:** DevOps is focused on improving time to market, lowering the failure rate of new releases, shortening the lead time between fixes, and minimizing disruption while maximizing reliability. To achieve this, DevOps organizations aim to automate continuous deployment to ensure everything happens smoothly and reliably. Companies that use DevOps methods benefit by significantly reducing time to market and improving customer satisfaction, product quality, and employee productivity and efficiency.
- **Cons:** Even in light of its benefits, there are a few drawbacks to DevOps:
- Some customers don’t want continuous updates to their systems.
- Some industries have regulations that require extensive testing before a project can move to the operations phase.
- If different departments use different environments, undetected issues can slip into production.
- Some quality attributes require human interaction, which slows down the delivery pipeline.

SW Development Methodologies: Top 4

Waterfall development method

Many consider the waterfall method to be the most traditional software development method. The waterfall method is a rigid linear model that consists of sequential phases (requirements, design, implementation, verification, maintenance) focusing on distinct goals. Each phase must be 100% complete before the next phase can start. There's usually no process for going back to modify the project or direction.



Source: https://www.synopsys.com/blogs/software-security/top-4-software-development-methodologies/

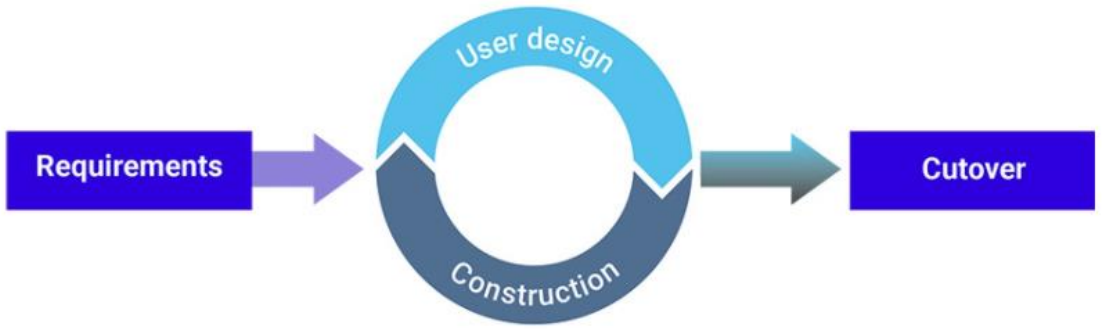
Waterfall Pros and Cons

- **Pros:** The linear nature of the waterfall development method makes it easy to understand and manage. Projects with clear objectives and stable requirements can best use the waterfall method. Less experienced project managers and project teams, as well as teams whose composition changes frequently, may benefit the most from using the waterfall development methodology.
- **Cons:** The waterfall development method is often slow and costly due to its rigid structure and tight controls. These drawbacks can lead waterfall method users to explore other software development methodologies.

SW Development Methodologies: Top 4

Rapid application development

- Rapid application development (RAD) is a condensed development process that produces a high-quality system with low investment costs. Scott Stiner, CEO and president of UM Technologies, said in Forbes, "This RAD process allows our developers to quickly adjust to shifting requirements in a fast-paced and constantly changing market." The ability to quickly adjust is what allows such a low investment cost.
- The rapid application development method contains four phases: requirements planning, user design, construction, and cutover. The user design and construction phases repeat until the user confirms that the product meets all requirements.



Source: https://www.synopsys.com/blogs/software-security/top-4-software-development-methodologies/

RAD Pros and Cons

- **Pros:** Rapid application development is most effective for projects with a well-defined business objective and a clearly defined user group, but which are not computationally complex. RAD is especially useful for small to medium projects that are time sensitive.
- **Cons:** Rapid application development requires a stable team composition with highly skilled developers and users who are deeply knowledgeable about the application area. Deep knowledge is essential in a condensed development timeline that requires approval after each construction phase. Organizations that don’t meet these requirements are unlikely to benefit from RAD.

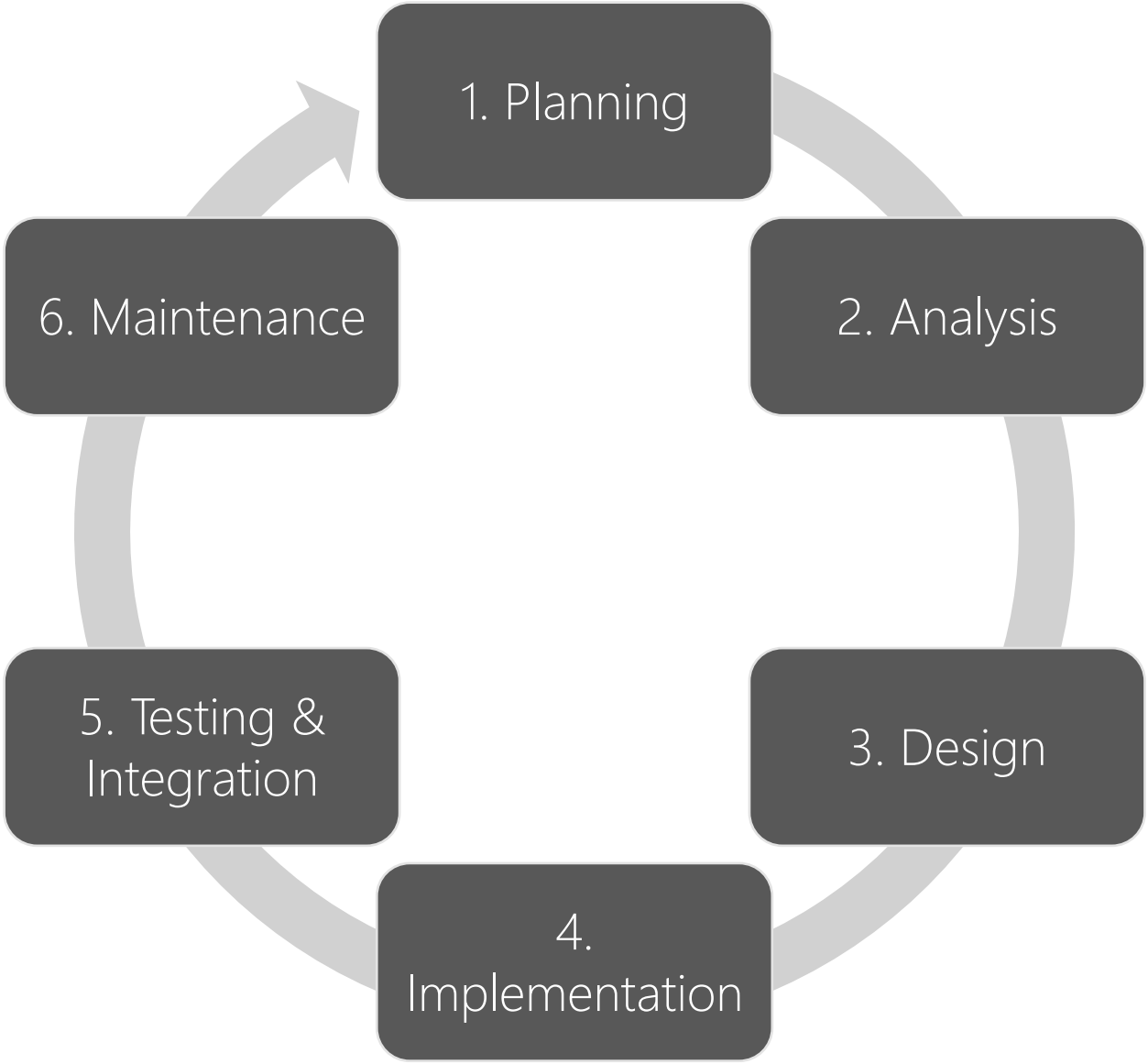
Software Engineering Processes (sometimes called SE lifecycle)

Important

- What it is: *Activities leading toward production of SW "product"*
- Relatively young field and not well understood nor manageable like other industrial production processes
- Involves great deal of judgment, creativity, but also experience and methodology.
- SW Engineering is somewhere between art and industrial process
- Efforts to automate and use "silver bullet tools or processes" failed
- It involves great deal of human interaction, teamwork and communication and is often messy
- Some successful *patterns and "best practices"* are emerging – need to know them (what they are and how and when to implement them)
- Increasingly asked in job interviews

- More critical for success of team projects than technology

Software Development Life Cycle (SDLC) / SE Process Lifecycle



SW Engineering Process Components

1. *Requirements and Specification*
2. *Design and Implementation*
3. *Validation*
4. *Maintenance and Evolution*

Important

SW Engineering Process Components

- There are several basic components present in all SW Engineering methodologies:
  1. *SW Requirements and Specification*: functionality, constraints, look and feel, GUI, performance
  2. *SW Design and Implementation*
  3. *SW Validation*: formal, test plans, QA
  4. *SW Maintenance and Evolution*: ongoing changes (and fixes) to meet customer needs

  These components are applied in different way and order (linear, iterative, spiral…) depending on chosen SW Engineering methodology or models (I.e. *waterfall, extreme, agile* etc.)

Important

SW Process Models (or Methodologies)

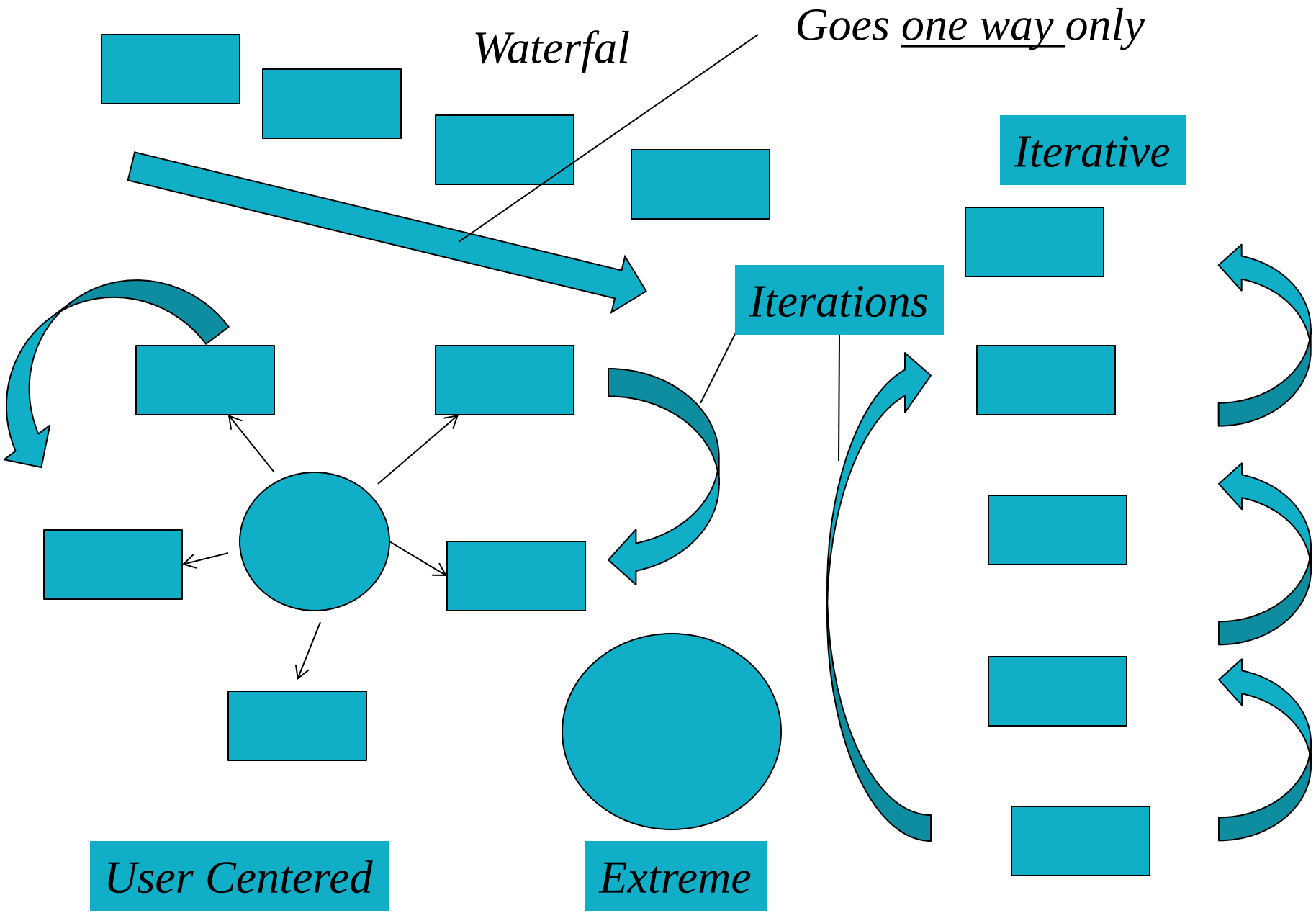
- Abstract representations of overall development of SW - methods and practices for organizing the SE effort
  - Waterfall
  - Evolutionary
  - Iterative
  - Open Source development
  - Extreme programming
  - For GUI and usability: User Centered Design (UCD) – variation of iterative development
  - Continuous development
- Agile, Scrum – high level project management (PM) processes used to "manage, organize and implement " the above SE processes

Methodologies

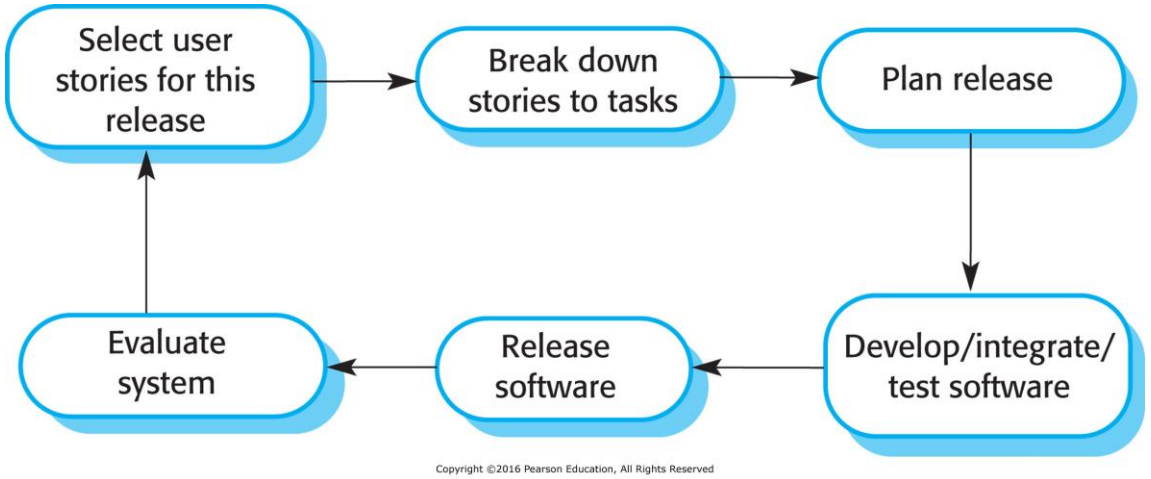
- Waterfall
- Evolutionary
- Iterative
- Incremental
- Open Source Development
- Extreme Programming
- User Centered Design (UCD)
- Continuous Development
- AGILE & SCRUM

Software process models

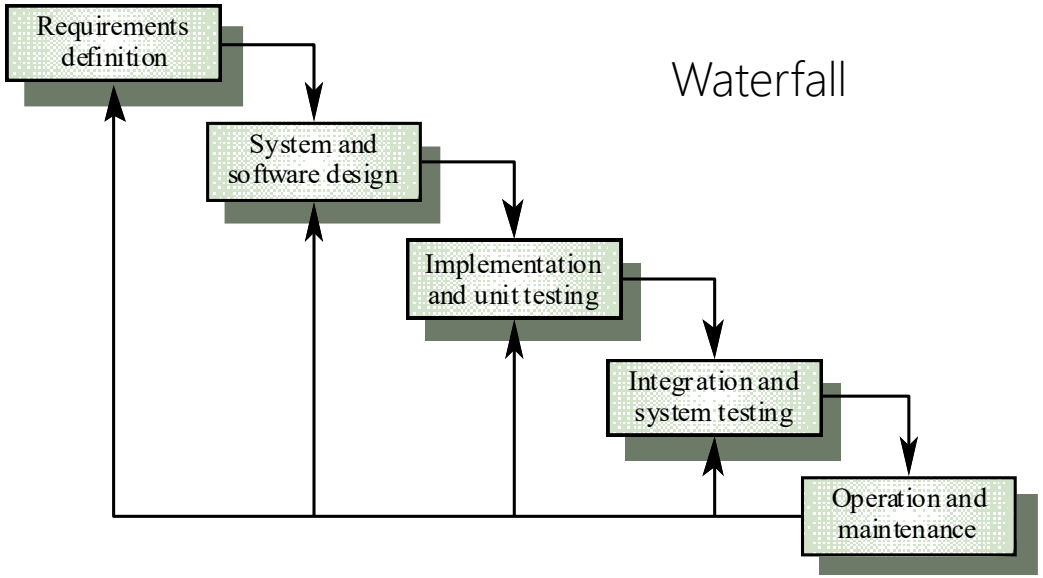
- The waterfall model
  - Plan-driven model. Separate and distinct phases of specification and development.
- Incremental development
  - Specification, development and validation are interleaved. May be plan-driven or agile.
- Integration and configuration
  - The system is assembled from existing configurable components. May be plan-driven or agile.
- In practice, most large systems are developed using a process that incorporates elements from all of these models.


Waterfal
Goes one way only
Iterative
Iterations
User Centered
Extreme

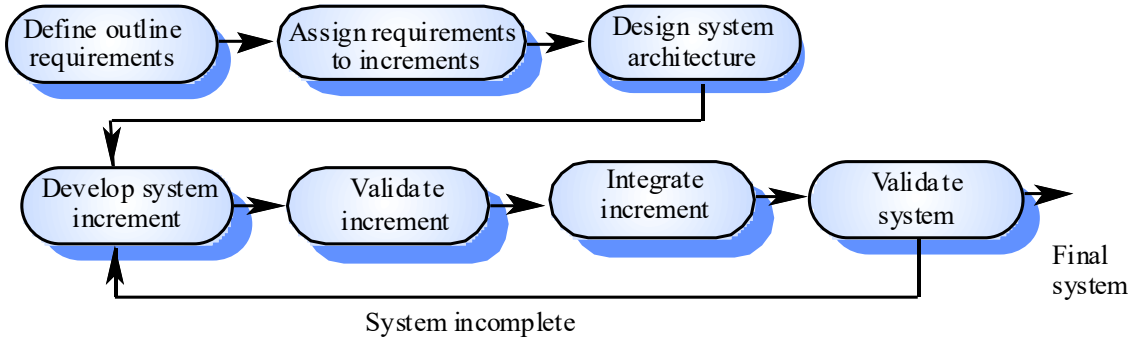
Extreme Programming



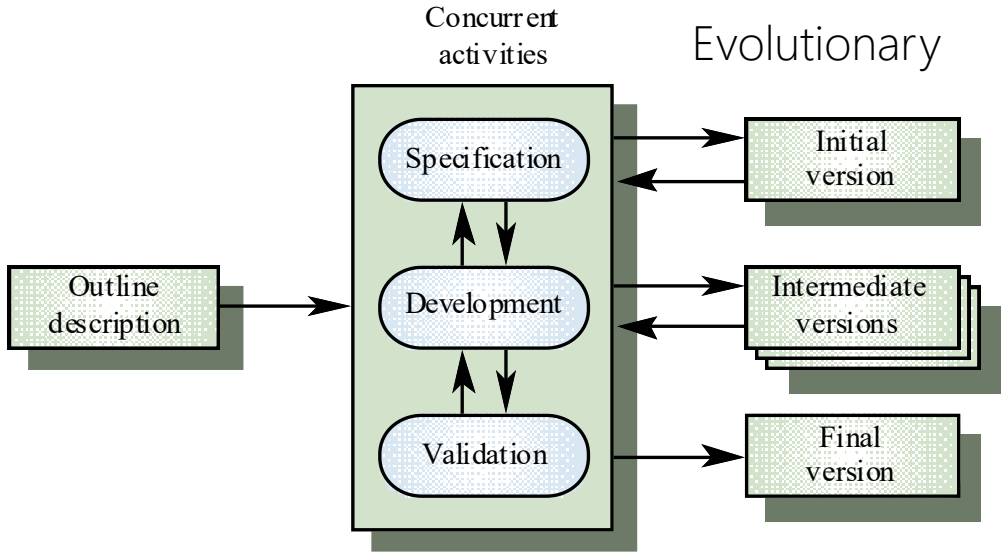
Waterfall



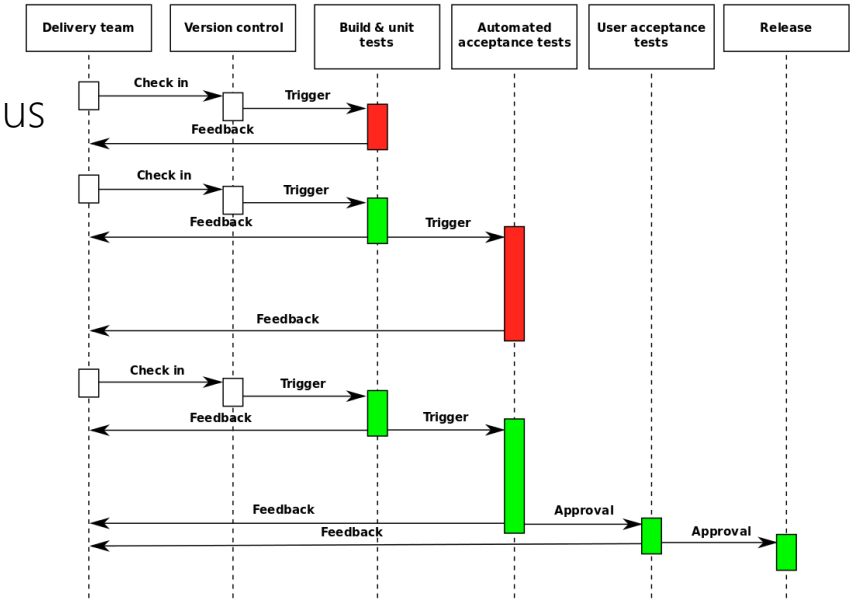
Incremental



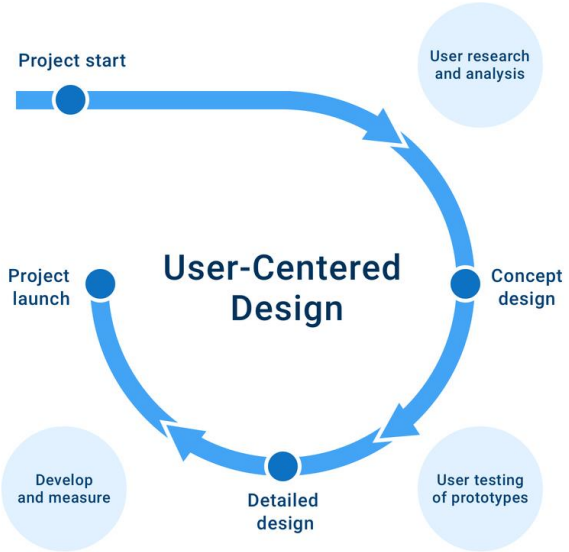
Evolutionary



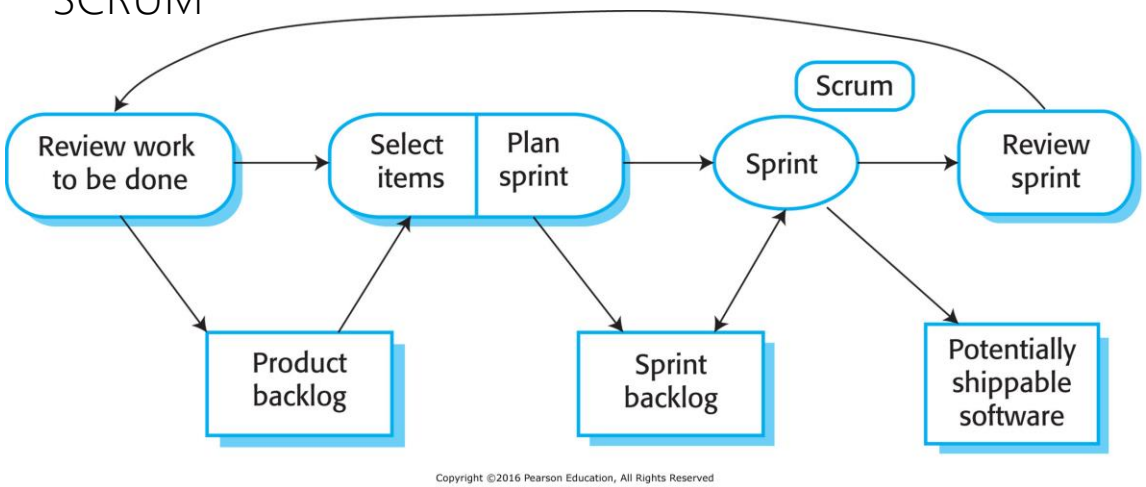
Continuous



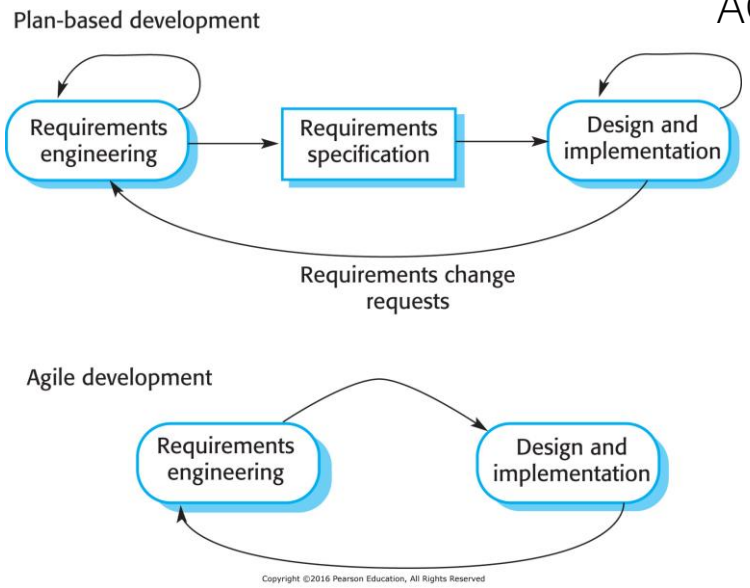
UCD



SCRUM

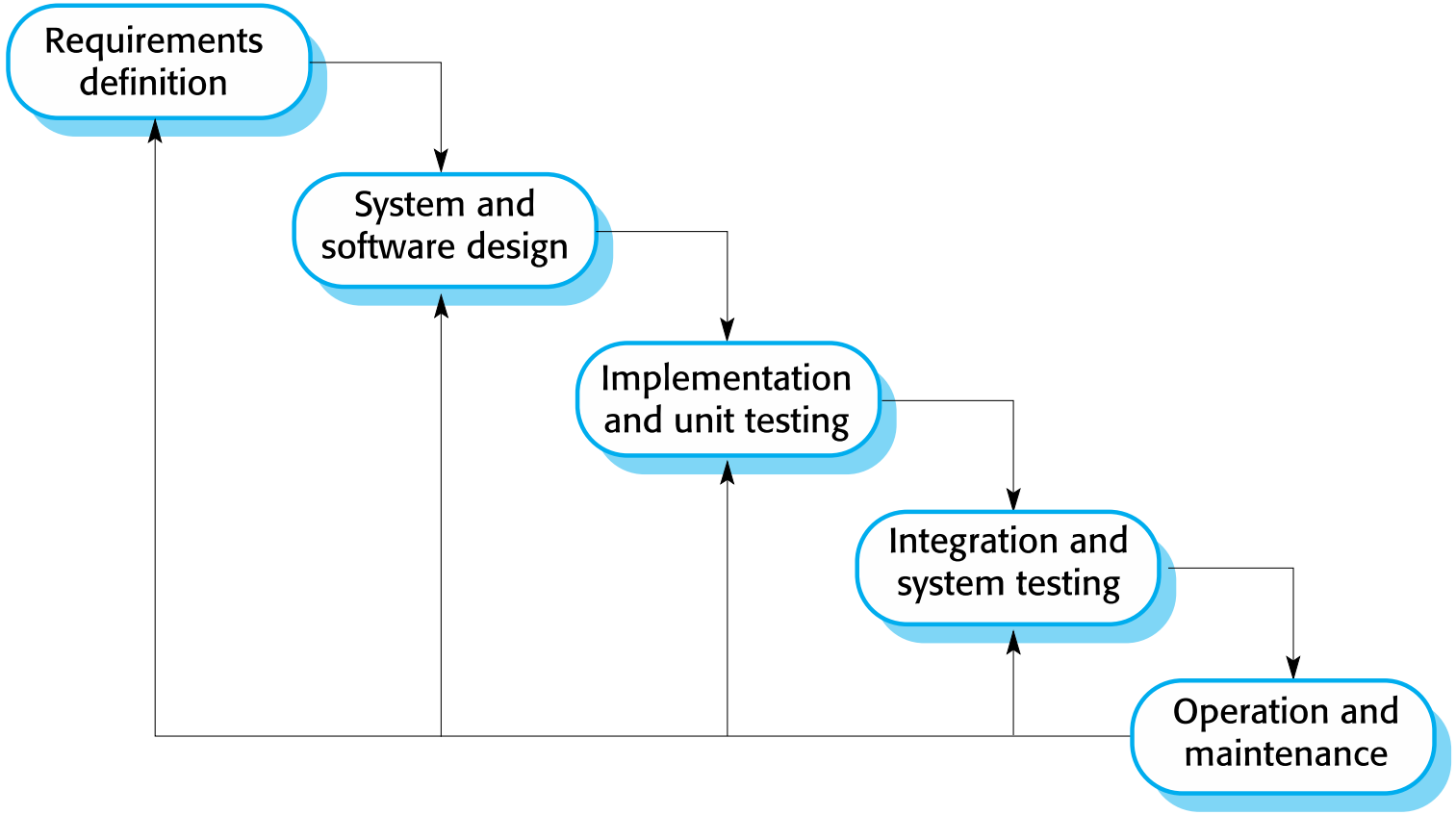


AGILE



The waterfall model

Important



Important

Waterfall Model

- It is the first published model (1970)
- Became basis of Military procedures
- Linear cascade from one step to another, no iterations
- It has distinct, formal phases, each resulting in formal documents, allowing easy management and tracking
- Iteration between phases are very limited
- <u>Key problem: inability to know full requirements and specs ahead of time</u>

<u>Seldom works, not much used today</u>

Waterfall and other models: some history

Important

- Late 1980 DOD review:
  - 75% of Waterfall projects failed or were never used. Only 2% were used
- 1995 Microsoft publicized daily build and smoke test, a very influential incremental and iterative SW development practice
- In 1994 and later DOD started replacing Waterfall standard with more flexible methods allowing iterative and incremental approach
- Iterative approaches recently called Agile SW Development, see the same IEEE issue devoted to this

Waterfall model phases

- There are separate identified phases in the waterfall model:
  - Requirements analysis and definition
  - System and software design
  - Implementation and unit testing
  - Integration and system testing
  - Operation and maintenance
- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. In principle, a phase has to be complete before moving onto the next phase.

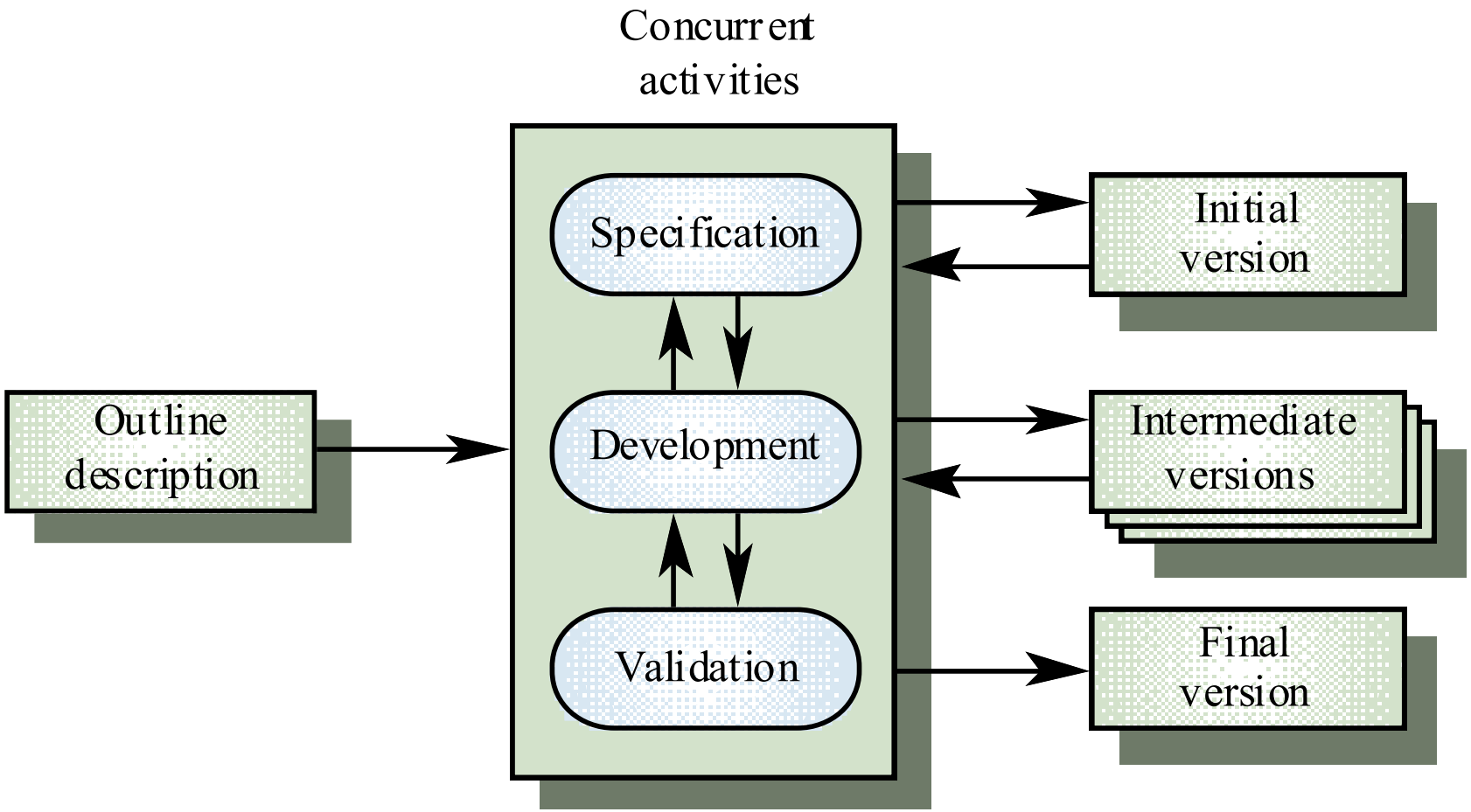
Waterfall model problems

- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
  - Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
  - Few business systems have stable requirements.
- The waterfall model is mostly used for large systems engineering projects where a system is devel
  - In those circumstances, the plan-d coordinate the work.

PROS	CONS
1. Clear goals and directions	1. Time consuming
2. Progress can be easily measured	2. Customer dissatisfaction
3. Saves time and money	3. Risk factor

Important

Evolutionary development



From: Ian Sommerwille

Important

Evolutionary development

- Develop initial implementation
- Expose it to users
- Iterate (requirements, design, implementation) until satisfactory result is obtained

- Rather than having separate specification, development and validation, these are carried out in parallel, with quick feedback and iteration

Important

Evolutionary development

- *Exploratory development*: work with customers starting from parts of the system that are well understood. Add new features proposed by customer until final product is delivered
- *Throw-away prototyping*: Experiment with customers using prototype. Results used to develop better and final system. Prototypes thrown away. Final system reimplemented

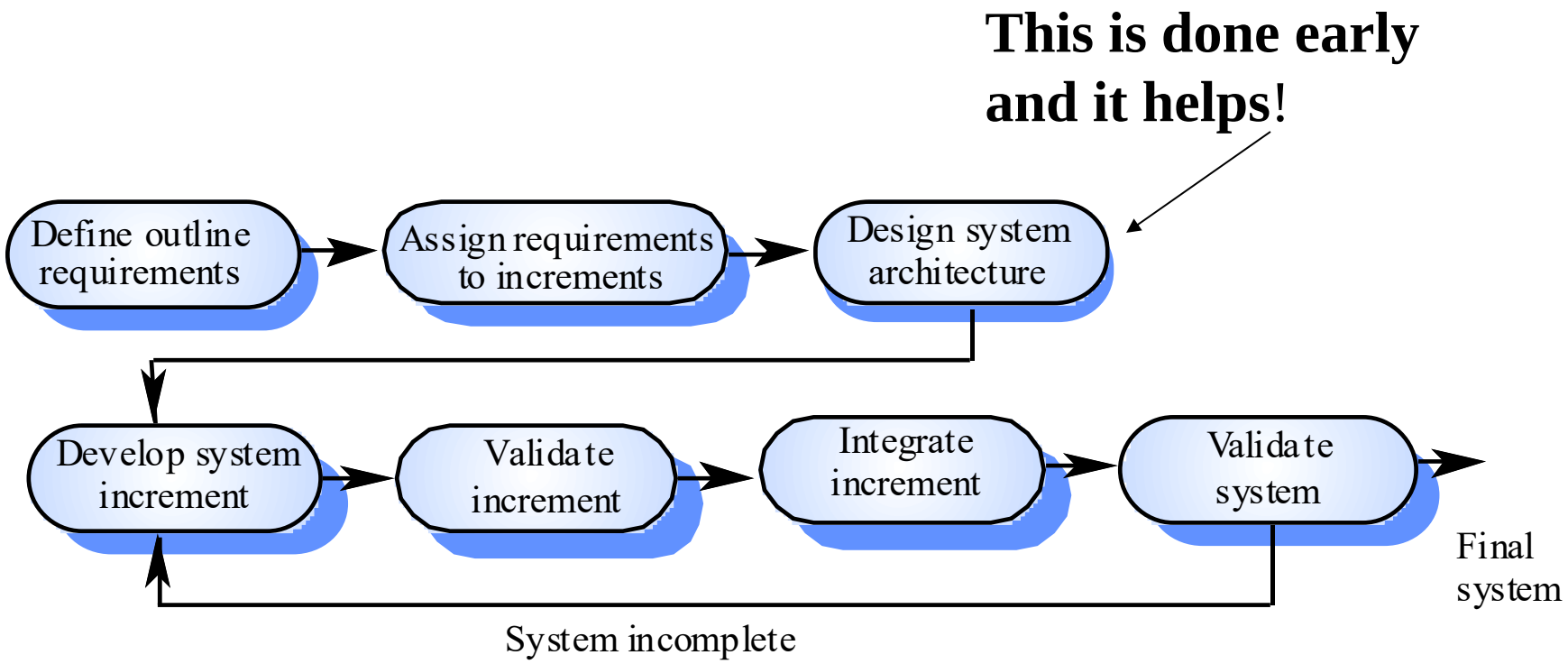
Important

Evolutionary development

- Issues and problems?
  - Can result in “spaghetti code and architecture” since increments are patched together from the start. OK in the beginning, but often results in poor scalability and maintenance
- Applicability?

Important

Incremental development



From: Ian Sommerville

Incremental Development

Important

- Waterfall model requires early commitment to requirements before design and before user experience. But, it is easy to manage and track.
- Evolutionary method helps in delaying some requirements until users get experience, but may result in poorly structured system because no general architecture is developed first

Incremental model is a compromise

- Customers identify increments in development and prioritize them.
- ➔Architecture is designed early with future requirements in mind
- Increments are developed and one by one integrated and validated. Each increment can use different SW Engineering model

Important

Incremental development

- Issues, problems?
- Applicability?
- Differences between this and evolutionary development?
- <u>Use Incremental (and Evolutionary) SE process (maybe with Extreme SE process) to develop projects from scratch and solicit revise requirements</u>

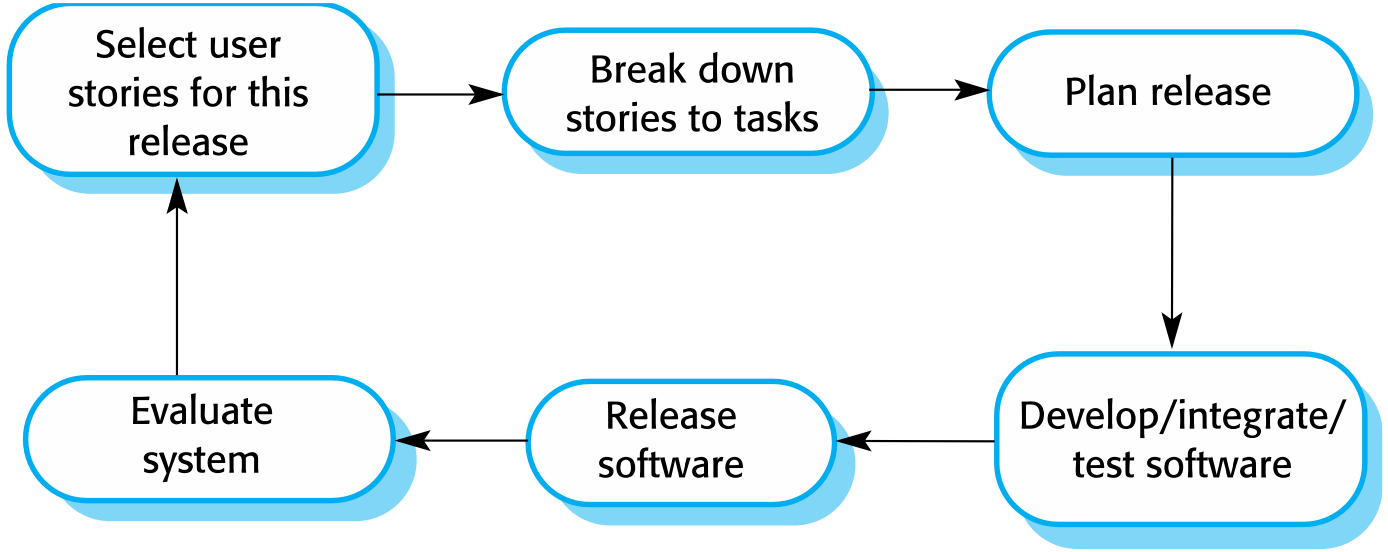
Incremental development benefits

- The cost of accommodating changing customer requirements is reduced.
  - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- It is easier to get customer feedback on the development work that has been done.
  - Customers can comment on demonstrations of the software and see how much has been implemented.
- More rapid delivery and deployment of useful software to the customer is possible.
  - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Incremental development problems

- The process is not visible.
  - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
- System structure tends to degrade as new increments are added.
  - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

The extreme programming release cycle



Important

eXtreme Programming (XP)

- A discipline of software development that is
  - lightweight, efficient, low-risk, flexible, predictable, scientific, and fun way to develop software
  - based on values of simplicity, communication, feedback and courage

XP and agile principles

- Incremental development is supported through small, frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People not process through pair programming, collective ownership and a process that avoids long working hours.
- Change supported through regular system releases.
- Maintaining simplicity through constant refactoring of code.

Important

eXtreme Programming (XP)

- Everything in XP is done in <u>small steps</u>:
  - Easy to plan
  - Easy to perform
  - Easy to evaluate
  - Easy to undo
- "Small steps" supports simplicity, feedback, lowered risks, predictability and flexibility

Extreme programming practices (a)

Principle or practice	Description
Incremental planning*	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'. See Figures 3.5 and 3.6.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring*	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Extreme programming practices (b)

Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

Practices of eXtreme Programming

- 13 practices of XP
  - Whole Team
  - Metaphor
  - Planning game
  - Simple design
  - Small releases
  - Customer tests
  - Pair programming
  - Test-driven development
  - Design improvement
  - Collective code ownership
  - Continuous integration
  - Sustainable pace
  - Coding standards

Important

Whole Team

- Remove artificial barrier between customer and development team
  - Client becomes a part of the team
  - Client makes critical decisions on the direction of the development team
- Win - Win

Important

Simple Design

- Design is always as simple as the current level of functionality allows
- NO extraneous complexity needed or allowed
- Keep the incremental design simple
  - Design must extend only far enough
  - Must encompass the next iteration's added features
- When code becomes cumbersome, it's time for refactoring

Important

Small Releases

- Tested & working code, whose functionality is specified by the client is very frequently released
- For every iteration, the client gets a new software
- Client evaluates & use the current status to dictate the content of the next delivery

Important

Collective Code Ownership

- Team as a whole owns the system software (use code reviews to achieve this…)
- Who originally wrote the code is most likely not remembered
- Anyone can modify any code at any time without asking for permission
- Revision control is definitely necessary
- With XP, you shouldn't expect your code to be in the same shape you left it and anything you find is fair game

Important

Continuous Integration

- New system builds multiple times per day
- At all times, regardless of level of functionality, system compiles, runs and passes all tests
- New code must again meet the same criteria

Important

Coding Standards

- Developed codes should not look like a mixture of different styles
- Team will adopt and follow some coding standard throughout development
- Particular standard is not nearly important as adhering to that standard

Important

Practices of eXtreme Programming

- 13 practices of XP
  - Whole Team
  - Metaphor
  - Planning game
  - Simple design
  - Small releases
  - Customer tests
  - Pair programming
  - Test-driven development
  - Design improvement
  - Collective code ownership
  - Continuous integration
  - Sustainable pace
  - Coding standards

Underlined: important components worth using

Important

When to use Extreme Programming

What teams:

- Very small teams, where everybody knows each other well

To do what:

- Early prototype, first SW version
- Demonstration of concepts to get feedback
- Technical validations, checking some feasibility

AND when timing is of essence

Important

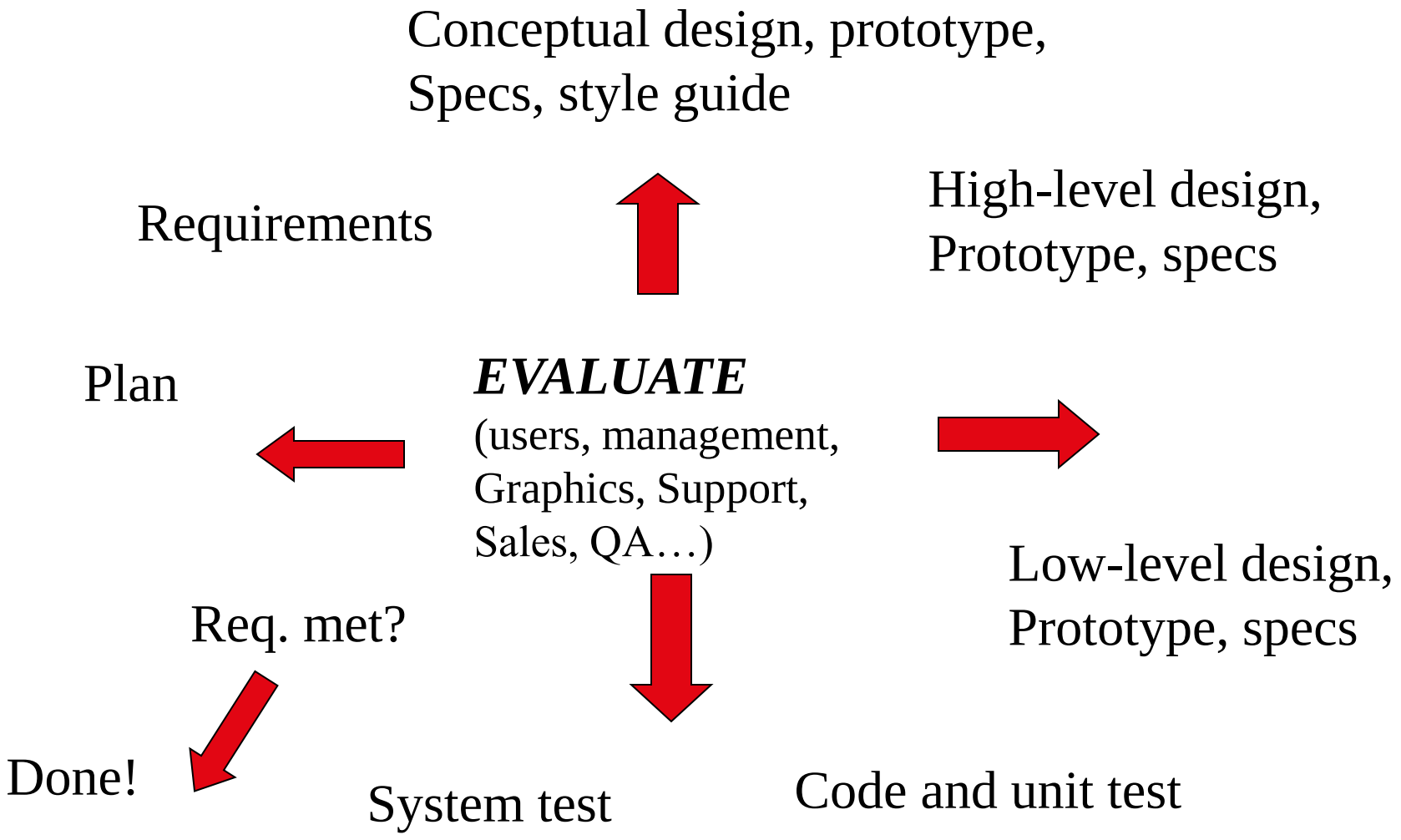
User Centered Design and Development for GUI (UCD)

- Usability and GUI design and development are the most people/user intensive area of SW engineering
- Know and listen to user, customer, client
- Iterate with user (and others) all the time, starting early
- GUI is a lot of code (often 50% of total), requires a lot of thinking, great detail and a lot of testing
- User involvement and constant iteration with users distinguish user-centered design & development from regular SW design & development
- GUI development is very rewarding: everybody sees quickly what you did (for better or worse ☺)!

User-Centered Design and Development –

Important

➔ <u>SE process of choice for achieving User Experience</u>



From: R. Torres

Important

Open Source Initiative

- All approved licenses must follow the Open Source Definition (excerpt from the 10 clauses):
  1. Free Redistribution
  2. Source Code available
  3. Derived Works allowed
  4. No Discrimination
- http://www.opensource.org/docs/osd

Conclusions on Open Source

Important

- Open Source communities can deliver industrial strength software
- Open Source will not replace Closed Source
- Different licenses emphasize different philosophies:
  - GPL: emphasizes community aspect
  - BSD: attribution of work is sufficient
  - MIT
- Companies are getting less afraid of using Open Source
- <u>Leverage it as much as you can (but make sure you test it in your applications and check the license ➔ integration testing in YOR specific app is critical)</u>

Important

Summary

- We presented various SE processes/models that guide application of basic commonest (requirements, design, implementation, testing…)
- This is your "tool box" – each method works best for some cases. No silver bullet (except maybe UCD for UI)
- In real application you will likely combine several methods

- But, how does one organize and manage such an activity at teamwork/project management level? This is addressed by higher level processes e.g. PM like Agile/SCRUM

Important

Agile SW Development Principles

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

The principles of agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Agile method applicability

- Product development where a software company is developing a small or medium-sized product for sale.
  - Virtually all software products and apps are now developed using an agile approach
- Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are few external rules and regulations that affect the software.

SCRUM Process – specific variant of Agile

Important

- About Scrum
  - http://en.wikipedia.org/wiki/SCRUM

- Tutorials:
  - http://www.wiziq.com/tutorials/scrum

Highly recommended – it works

Important

SCRUM summary

- Most effective way to manage SW teams, widely used as a way to <u>manage SW development</u> especially in small and medium companies
- Inner loop – daily, weekly "increments"- going over task list in a short meeting.
  - Key questions SCRUM organizer asks every day
    1. What have you done since last meeting
    2. What you plan to do in immediate future
    3. Problems, issues, help needed?
- Outer loop – once a moth or once a week – strategy development and check of overall progress
- Familiar?
- <u>Very effective, use it !!!!!!!!</u>

Important

Continuous Development/Delivery/Deployment

- Part of Agile SE Process and Extreme Programming methods
  - The Agile Principle
    "Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale."
- Also from Extreme programming
  - Deliver everything in small steps
  - "Small steps" supports simplicity, feedback, lowered risks, predictability and flexibility

Important

Motivation

A) Traditional:

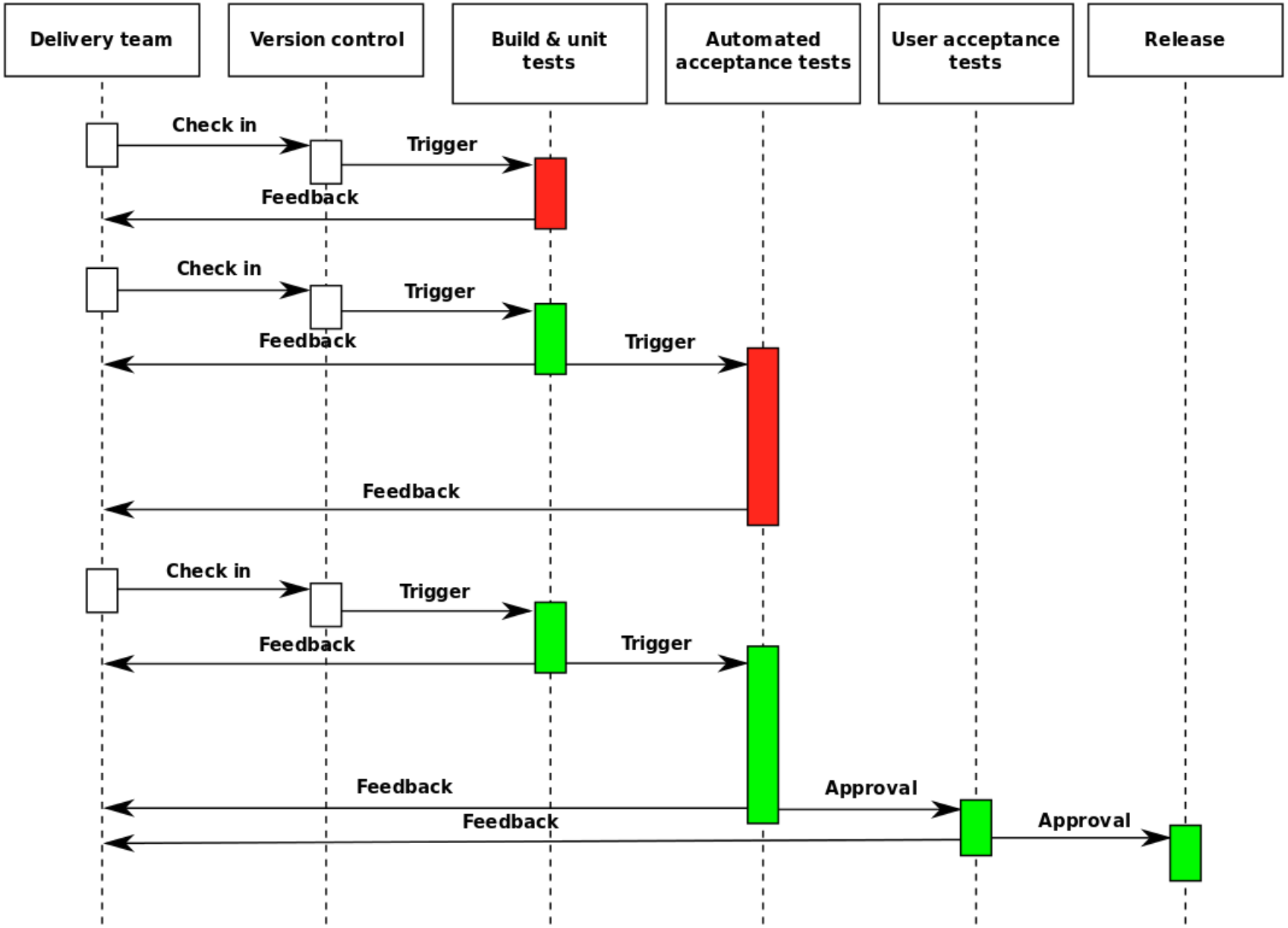
- Last build done 2 weeks ago, it worked then.
- 125 updates/changes made since.
- Current build does not work.
- Client coming to see the demo tomorrow.

B) Continuous development:

- Builds done daily with smoke test.
- If tests fail code not added to repo until errors resolved.
- Since yesterday's working build 5 updates made.
- Client evaluated working demo 3 times a week for last month.

Which one you would rather be managing?

https://en.wikipedia.org/wiki/Continuous_delivery



Rubrics for team project grading

Measured Team Outcomes for SE *process*	**Measured Team Outcomes for SE *product***
1. Fraction of the team participating at the meetings with the instructor 2. Quality and timing of follow-up on outstanding issues 3. Ability to deal with feedback constructively 4. Producing the non-SW and SW deliverables on time; following submission guidelines 5. Quality and completeness of non-SW deliverables (requirements, use cases, UI mockups, design documents, test plans etc.) as in M5 folder 6. Number of teamwork issues that instructor had to deal with (whether resolved or not) 7. Ability to apply best SE process and teamwork practices 8. Adherence to continuous and collaborative development practices 9. Ability to effectively use the SE tools (e.g. collaborative tools, version control, issue trackers)	1. Correctness and reliability of operation 2. Functionality of the product vs. desired requirements and use cases 3. Ease of use, user interface 4. Architecture of the developed system 5. Database design 6. Basic security, field validation 7. Performance 8. Code quality and “smell” (coding principles, style, documentation) 9. Proper usage of collaborative tools e.g. repository 10. Presentation style and effectiveness of final product demonstration (content, delivery, adherence to time, dealing with Q/A)
25 %	25 %

Product Demo (M5)

Milestone 5 is the <u>final delivery of your project</u> and provides basis for grading it. It consists of

1. **Demo and presentation of your application to the class**
2. **Delivery of M1-M5 documentation in the "M5 Folder" – complete final project documentation soft copy .DOC or .PDF files in the corresponding github folder**

- 06:10-06:25PM – Team #
- 06:30-06:45PM – Team #
- 06:50-07:05PM – Team #
- 07:10-07:25PM – Team #
- 07:30-07:45PM – Team #
- 07:50-08:05PM – Team #

Milestone 5

- Final Demo and Documents for Grading (Due date: December 10th – by 5PM). NOT EXCEPTIONS!
  - Delivery 1: Demo the Application (15 min).
  - Delivery 2: Folder with complete project documentation – "M5 folder"
    - e-mail subject line: Must be "CSC648-848 Fall 2024 Section 2 Milestone5 Folder Team N" in the subject line (N is a team number).
    - e-mail body contains direct link to Milestone 5 folder in team github

    1. Cover Page
    2. Product Summary
    3. Milestone Documents (M1-M4)
    4. Screenshots of actual final product as shown in the demo
    5. Screenshots of key DB tables
    6. Google analytics plot for your WWW site
    7. Screenshots of your task management system (like Trello) showing a snapshot of your project Management
    8. Team Members contributions
    9. Post analysis – lessons learned

- Schedule (Tuesday Dec 10th):
  - 06:10-06:25PM – Team #
  - 06:30-06:45PM – Team #
  - 06:50-07:05PM – Team #
  - 07:10-07:25PM – Team #
  - 07:30-07:45PM – Team #
  - 07:50-08:05PM – Team #

Individual Class Reflection Paper

1. Analysis of the entire Software Engineering class
2. Experience working with the team project
3. Innovation Space
4. Resume
5. Preferred Job and Plan to enter the High Tech and Job Market as Computer Scientist or any other position.

Due Date: Dec 10th **by 5PM** NOT EXCEPTIONS!

M4-5

- **Milestone 4.** Beta Launch, QA and Usability Testing and Final Commitment for Product Features (P1 list). November 12th 2024 (W12)

- **Milestone 5.** Final Demo and Documents for Grading. December 10th 2024 (W15)

Thank You